Relational Database Management System (RDBMS)

240110102

Definitions

- Data: (Plural of Datum – Single piece of Information)
  - Data is a known fact
  - Can be recorded and have implicit meaning
  - Raw or isolated facts from which the required information is produced
  - Data can exist in a variety of forms that have meaning in the user's environment
  - Data can be an object such as documents, images, video segments etc…

- Example: A person's Name
  - In an organization it is Employee Name
  - In an institute it is Student Name
  - In a Bill it is Customer Name
  - In a firm it is Client Name

Information

- Information is processed, organised or summarised data
- It is defined as a collection of related data, that when put together, communicate meaningful and useful message to a recipient who use it to make decision.

Database and Database System

- Database is a kind of electronic filing cabinet; It is a repository or a container for a collection of computerised data files.
- Database is a collection of persistent data that is used by the application systems of some given enterprise (eg. Manufacturing company, Bank, Hospital, university, Govt. Dept. etc…)
- Database system is just a computerised record keeping system
- Users of the database system can perform variety of operations as
  - Adding new files to the database
  - Inserting data into existing files
  - Retrieving data from the existing file
  - Deleting data from the existing file
  - Modifying data in the existing file
  - Removing existing files from the database

Disadvantages of File-Oriented System

- Data redundancy
- Data inconsistency
- Program – Data dependence
- Poor data control
- Limited data sharing
- Inadequate data manipulation capabilities
- Excessive programming efforts
- Security problems

Functions/ Advantages of DBMS

- **Controlling Redundancy**: Data redundancy leads to wasted storage space, duplication of effort (when multiple copies of a datum need to be updated), and a higher liklihood of the introduction of inconsistency
- **Restricting Unauthorized Access:** provide a security and authorization subsystem, which is used for specifying restrictions on user accounts. Common kinds of restrictions are to allow read-only access (no updating), or access only to a subset of the data
- **Providing Persistent Storage for Program Objects**:
- **Providing Storage Structures for Efficient Query Processing**: The DBMS maintains indexes (form of trees and/or hash tables) that are utilized to improve the execution time of queries and updates.
- **Providing Backup and Recovery:** The subsystem having this responsibility ensures that recovery is possible in the case of a system crash during execution of one or more transactions.
- **Providing Multiple User Interfaces:**
- **Representing Complex Relationships Among Data:**
- **Enforcing Integrity Constraints:**

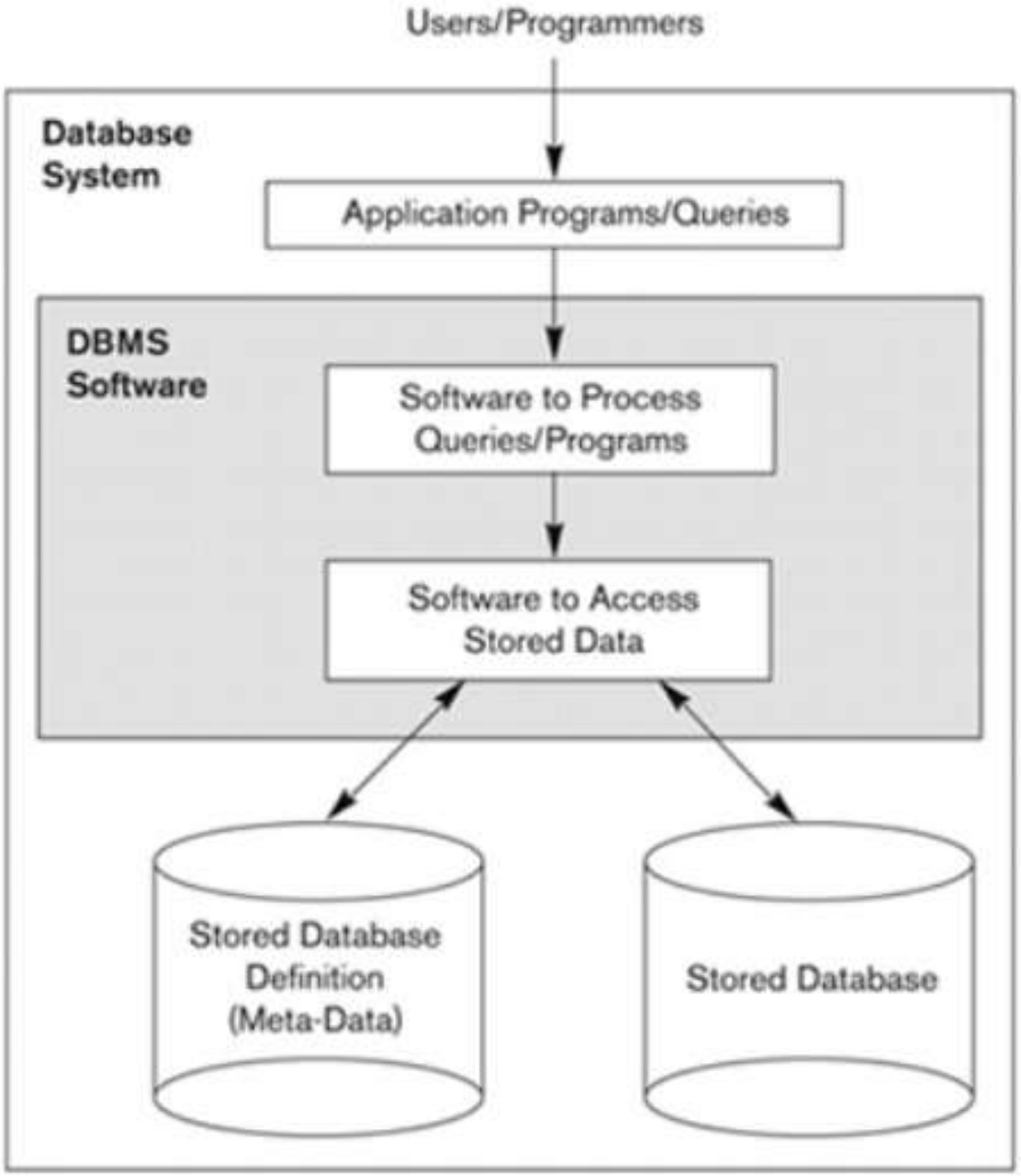
Difference between File System and Database Management System (DBMS)

Feature	File System	Database Management System (DBMS)
1. Definition	A file system is a method for storing and organizing files on storage devices like HDDs or SSDs.	A DBMS is a software system that enables creation, manipulation, and management of databases.
2. Data Storage	Data is stored in separate files, usually in formats like .txt, .csv, etc.	Data is stored in structured tables in a database.
3. Data Access	Manual access or through basic programming logic (e.g., file handling in Python).	Accessed using structured queries (e.g., SQL).
4. Data Redundancy	High — same data may be stored in multiple files.	Controlled through normalization and relational design.
5. Data Integrity	Hard to enforce; handled manually in the application code.	Enforced through constraints, keys, and rules in DBMS.
6. Security	Basic file-level security (read/write permissions).	Advanced user-level and object-level security controls.
7. Concurrency	Difficult — simultaneous access may corrupt data.	Managed through transactions, locks, and concurrency control.
8. Backup and Recovery	Needs manual handling via system tools or scripts.	Built-in backup and recovery options in DBMS.
9. Querying	No support for complex queries — needs procedural code.	Supports powerful query languages like SQL.
10. Scalability	Not scalable for large or complex datasets.	Scalable and efficient for large enterprise applications.

Relational Database Management System (RDBMS)

240110102

Simplified database system environment



Database Environment

- Database environment is a collective system of components that comprises and regulate the group of data, management and use of data which consists of s/w, h/w, people, techniques of handling DB and data also.
- DBMS: A piece of s/w that manages databases and lets you create, edit and delete databases, their tables and their data.
- DBMS are of 3 type:
  - RDBMS: Stores its data and metadata in tables and access them using SQL
  - OODBMS: Represent the data in the form of objects and uses OQL
  - ORDBMS: Represents some part of the data in the form of tables and some other part of the data in the form of objects. It uses SQL3 which is a combination of SQL2 and OQL
- Example of RDBMS
  - Oracle, DB2, IBM, MySQL, Microsoft SQLServer and PostgreSQL, SAP, Sybase ASE, Informix etc…

Characteristics of Database Approach

- Self-describing nature of a database system (Metadata)
- Insulation between programs and data – Data Abstraction (Program data Independence)
- Supports multiple views of the data
- Sharing of data and multiuser transaction processing

Data Models

- Underlying the structure of a database is the data model
- It is a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints
- A data model provides a way to describe the design of a database at the physical, logical, and view levels
- There are a number of different data models that can be classified into four different categories
  - (Network data model and the Hierarchical data model – are now obsoleted model, only old database code uses it)
  - Relational model
  - ER Model
  - Object Based Data Model
  - Semi structured Data Model

Relational Model

- The relational model uses a collection of tables to represent both data and the relationships among those data.
- Each table has multiple columns, and each column has a unique name.
- Tables are also known as relations.
- The relational model is an example of a record-based model.
- Record-based models are so named because the database is structured in fixed-format records of several types.
- Each table contains records of a particular type.
- Each record type defines a fixed number of fields, or attributes.
- The columns of the table correspond to the attributes of the record type.
- The relational data model is the most widely used data model, and a vast majority of current database systems are based on the relational model

Entity-Relationship Model

- The entity-relationship (E-R) data model uses a collection of basic objects, called entities, and relationships among these objects.
- An entity is a “thing” or “object” in the real world that is distinguishable from other objects.
- The entity-relationship model is widely used in database design

Data dictionary

- The data dictionary is considered to be a special type of table that can only be accessed and updated by the database system itself (not a regular user).
- The database system consults the data dictionary before reading or modifying actual data
- Data dictionary contains Meta Data about the structure of the database, i.e. the schema of the database.
- A relational database system needs to maintain data about the relations, such as the schema of the relations. In general, such “data about data” is referred to as metadata
- Data dictionary is also called as System Catalog
- The data dictionary is often stored in a nonnormalized form to achieve fast access

Information stored in Data Dictionary are:

- Names of the relations.
- Names of the attributes of each relation.
- Domains and lengths of attributes.
- Names of views defined on the database, and definitions of those views.
- Integrity constraints (for example, key constraints).
- Names of authorized users.
- Authorization and accounting information about users.
- Passwords or other information used to authenticate users.
- Number of tuples in each relation.
- Method of storage for each relation
- If relations are stored in operating system files, the dictionary would note the names of the file (or files) containing each relation.
- If the database stores all relations in a single file, the dictionary may note the blocks containing records of each relation in a data structure such as a linked list.
- Name of the index.
- Name of the relation being indexed.
- Attributes on which the index is defined.
- Type of index formed.

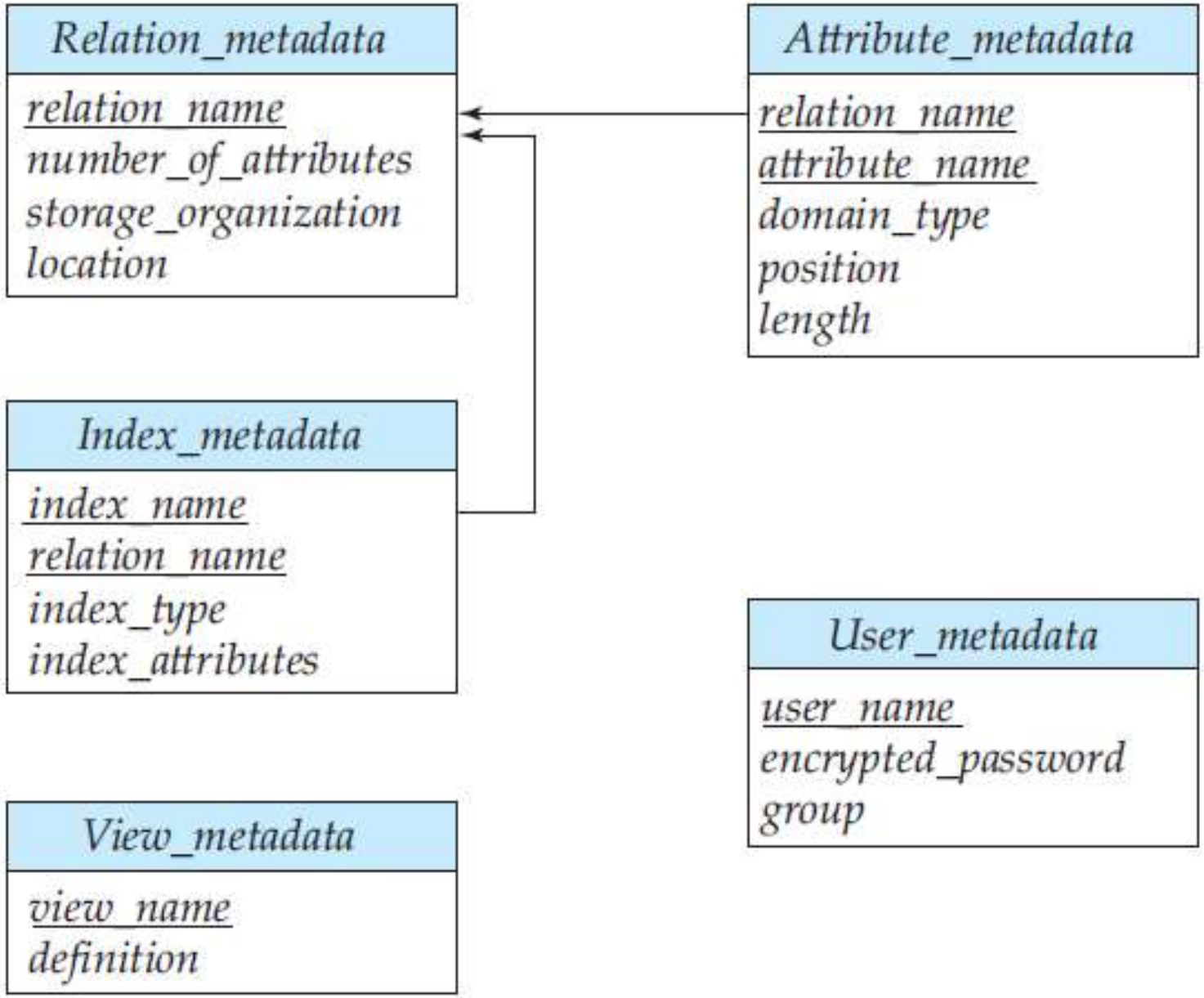


Figure 10.16 Relational schema representing system metadata.

Activ
Go to

Types of Data Dictionary

- There are two types of data dictionaries: active and passive
- Active data dictionary:
  - Any changes to the database object structure via DDLs will have to be reflected in the data dictionary. But updating the data dictionary tables for the changes are responsibility of database in which the data dictionary exists.
  - Active data dictionary is created in the same database, so the DBMS software will automatically update the data dictionary. Hence there will not be any mismatch between the actual structure and the data dictionary details.
  - The modification is an automatic task and most RDBMS has active data dictionary. It is also known as integrated data dictionary.
- Passive data dictionary:
  - Data dictionary created separately from the current database as entirely new database to store only data dictionary's information. Sometimes it is stored as xml, excels or in any other file format. In such case, an effort is required to keep data dictionary in sync with the database objects. This kind of data dictionary is called passive data dictionary. In this case, there is a chance of mismatch with the database objects and the data dictionary. This kind of DD has to be handled with utmost care.
  - This data dictionary is not a choice for many users as it is not easy to maintain
  - Managed by the users and is modified manually when the database structure change. Also known as non-integrated data dictionary.

Relational Database Management System (RDBMS)

240110102

Database Users

- Database Administrator (DBA) :
- Naive / Parametric End Users :
- System Analyst :
- Sophisticated Users :
- Data Base Designers :

Navie User

- Navie users are unsophisticated users who interact with the system by invoking one of the application programs that have been written previously.
- For example, a clerk in the university who needs to add a new instructor to a department.
- The typical user interface for navie users is a forms interface, where the user can fill in appropriate fields of the form.
- Navie users ay also simply read reports generated from the database.

Application programmers

- Application programmers are computer professionals who write application programs.
- Application programmers can choose from many tools to develop user interfaces.
- Eg. Rapid application development (RAD) tools are tools that enable an application programmer to construct forms and reports with minimal programming effort.

Sophisticated users

- Sophisticated users interact with the system without writing programs.
- Instead, they form their requests either using a database query language or by using tools such as data analysis software.
- Analysts who submit queries to explore data in the database fall in this category
- Sophisticated users can be engineers, scientists, business analyst, who are familiar with the database

Specialized Users

- Users who write specialized database applications
- Among these applications are computer-aided design systems, knowledgebase and expert systems, systems that store data with complex data types (for example, graphics data and audio data), and environment-modeling systems

Data Base Designers

- Data Base Designers are the users who design the structure of data base which includes tables, indexes, views, constraints, triggers, stored procedures.
- He/she controls what data must be stored and how the data items to be related.

Database Administrator

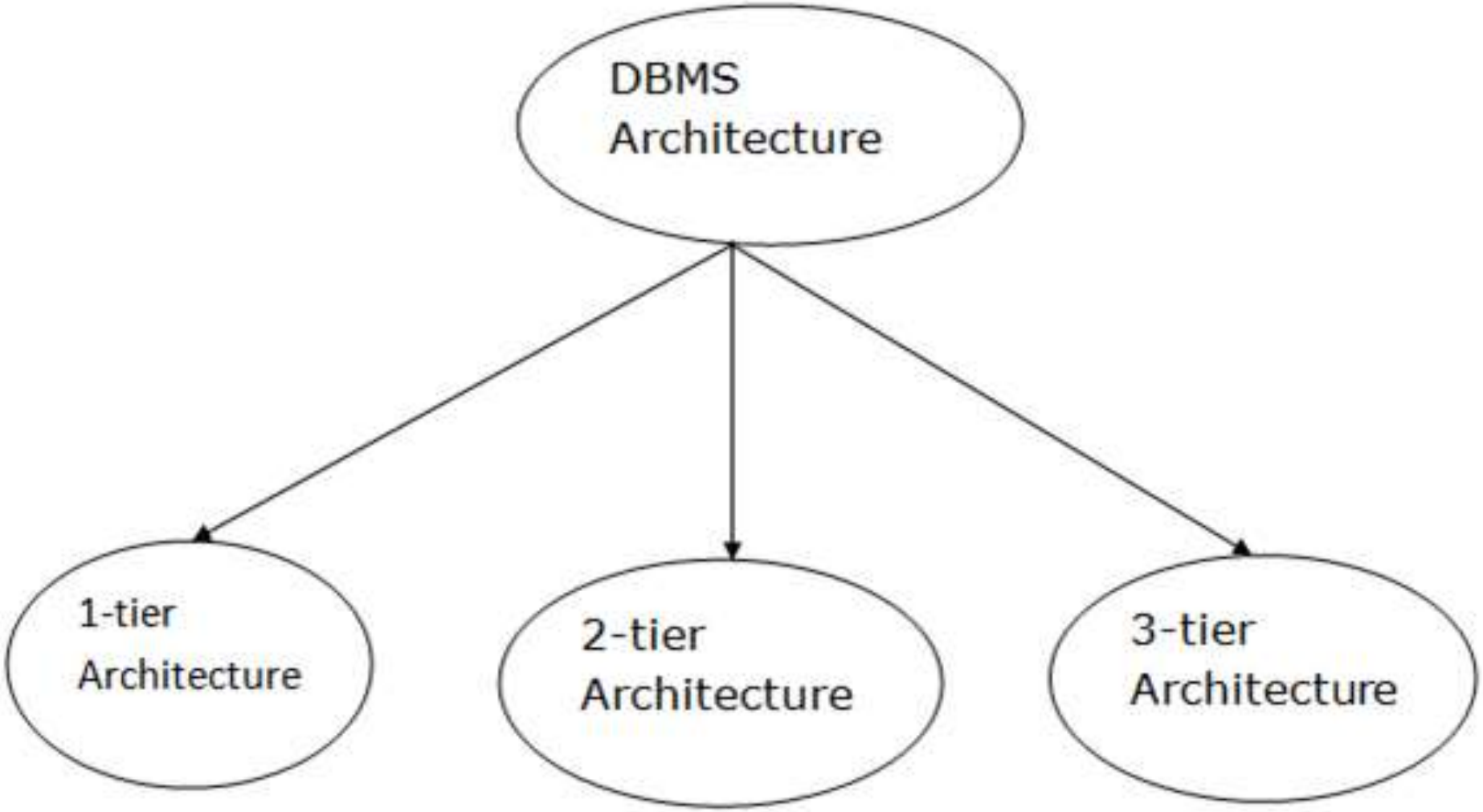
- A person who has such central control of both the data and the programs that access those data is called a database administrator (DBA).
- The functions of a DBA include:
  - Schema definition. The DBA creates the original database schema by executing a set of data definition statements in the DDL.
  - Storage structure and access-method definition.
  - Schema and physical-organization modification.
  - Granting of authorization for data access.
  - Routine maintenance. Examples of the database administrator's routine maintenance activities are:
    - Periodically backing up the database, either onto tapes or onto remote servers, to prevent loss of data in case of disasters such as flooding.
    - Ensuring that enough free disk space is available for normal operations, and upgrading disk space as required.
    - Monitoring jobs running on the database and ensuring that performance is not degraded by very expensive tasks submitted by some users.

Relational Database Management System (RDBMS)

240110102

Database Architecture

- The DBMS design depends upon its architecture. The basic client/server architecture is used to deal with a large number of PCs, web servers, database servers and other components that are connected with networks.
- The client/server architecture consists of many PCs and a workstation which are connected via the network.
- DBMS architecture depends upon how users are connected to the database to get their request done.

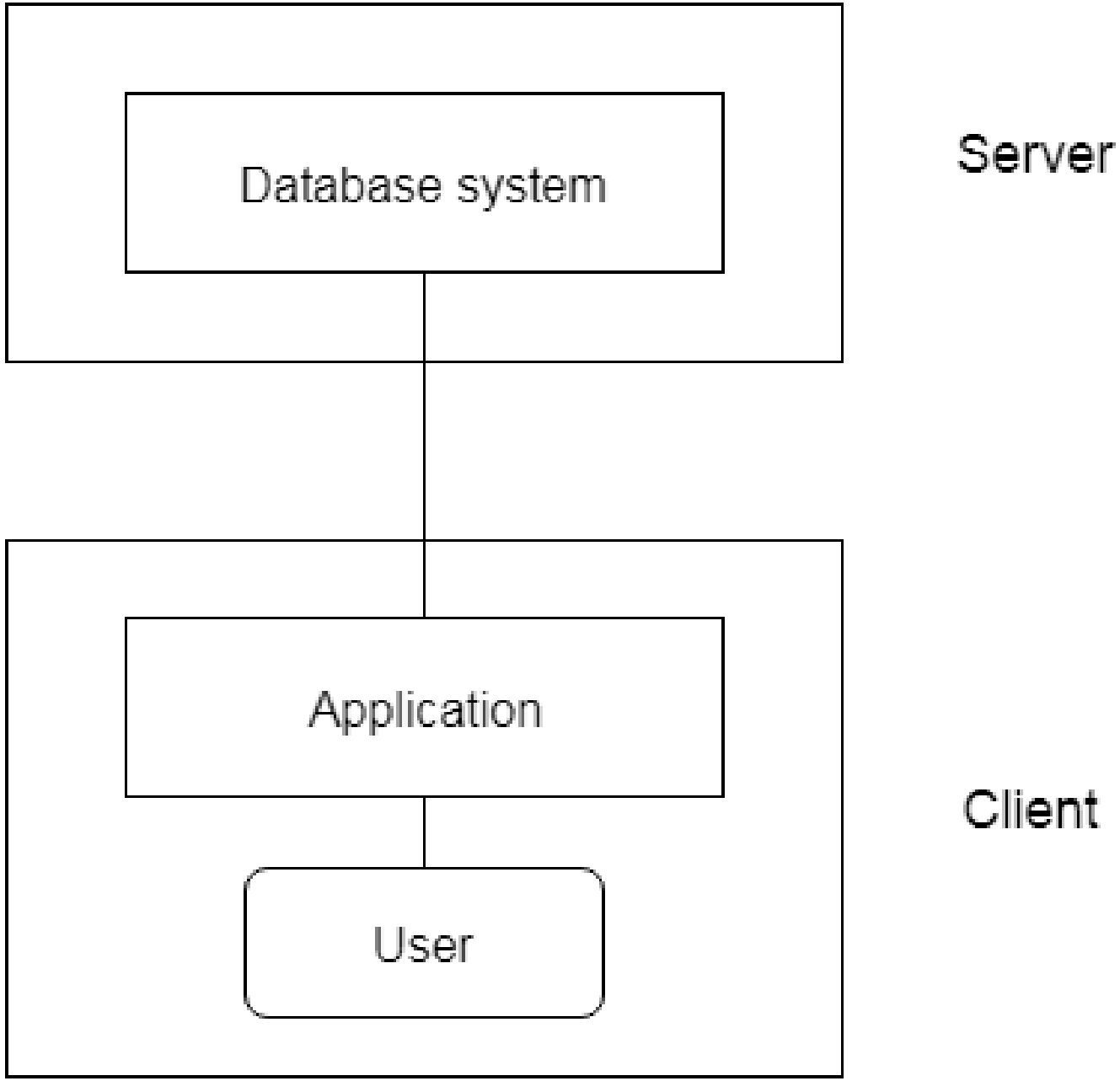

DBMS
Architecture
1-tier
Architecture
2-tier
Architecture
3-tier
Architecture

1-Tier Architecture

- In this architecture, the database is directly available to the user. It means the user can directly sit on the DBMS and uses it.
- Any changes done here will directly be done on the database itself. It doesn't provide a handy tool for end users.
- The 1-Tier architecture is used for development of the local application, where programmers can directly communicate with the database for the quick response.
- Advantages:
  - simplicity, cost-effectiveness, and ease of deployment,
  - Particularly used for small-scale or personal databases.
  - It's also known for faster data access for a single user due to the absence of network communication overhead
- Limitations:
  - lacks scalability, challenges in data sharing
  - support a single user at a time, making it unsuitable for most modern applications.

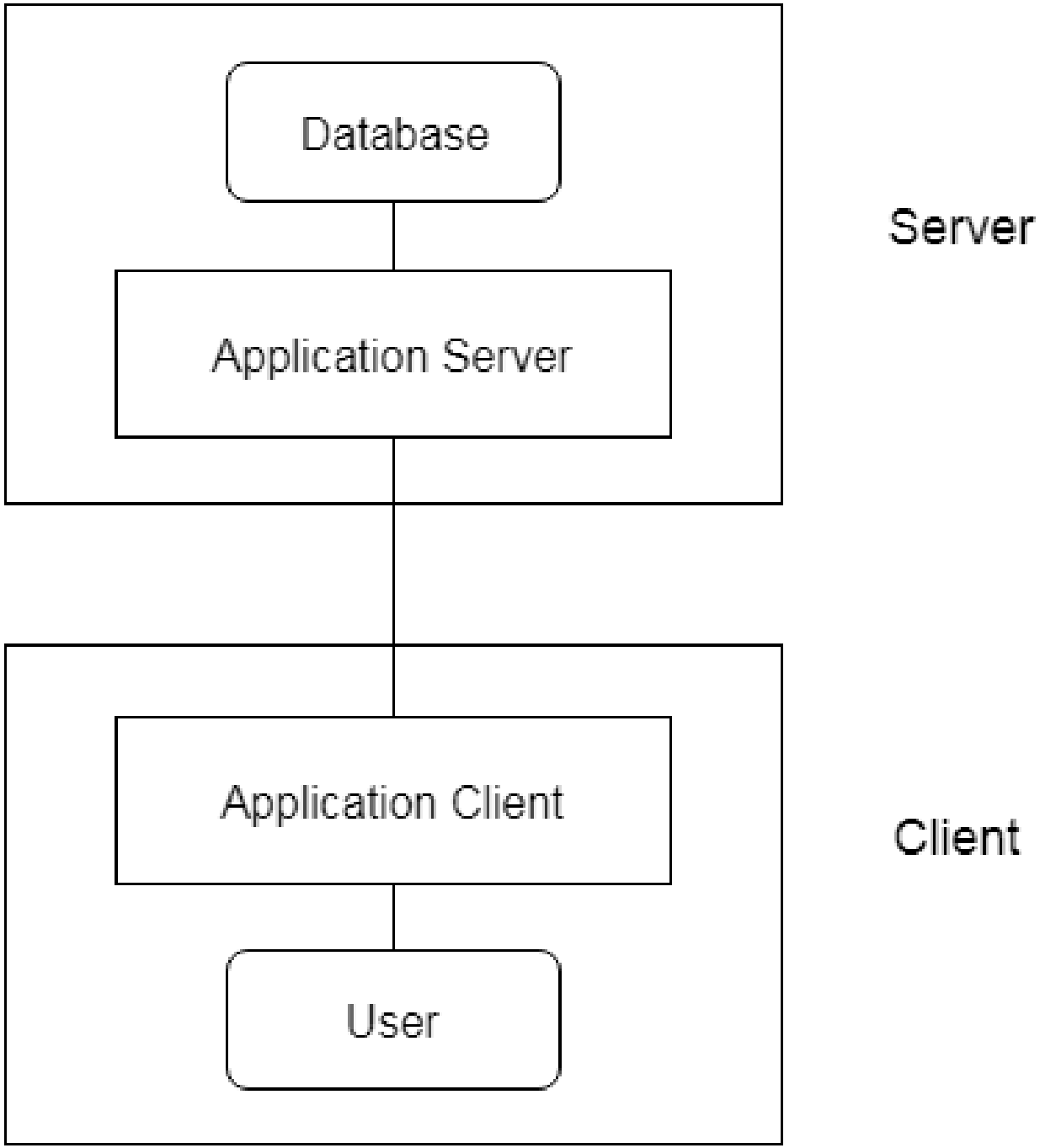
2-Tier Architecture

- The 2-Tier architecture is same as basic client-server. In the two-tier architecture, applications on the client end can directly communicate with the database at the server side. For this interaction, API's like: ODBC, JDBC are used.
- The user interfaces and application programs are run on the client-side.
- The server side is responsible to provide the functionalities like: query processing and transaction management.
- To communicate with the DBMS, client-side application establishes a connection with the server side.
- Advantages:
  - simplicity and direct client-server communication.
  - Key benefits include ease of access, scalability, cost-effectiveness, and simple deployment
- Limitations:
  - scalability, security, and maintainability.
  - It's generally not suitable for large, complex applications with many users due to performance bottlenecks and security vulnerabilities.


Database system
Server
Application
User
Client

3-Tier Architecture

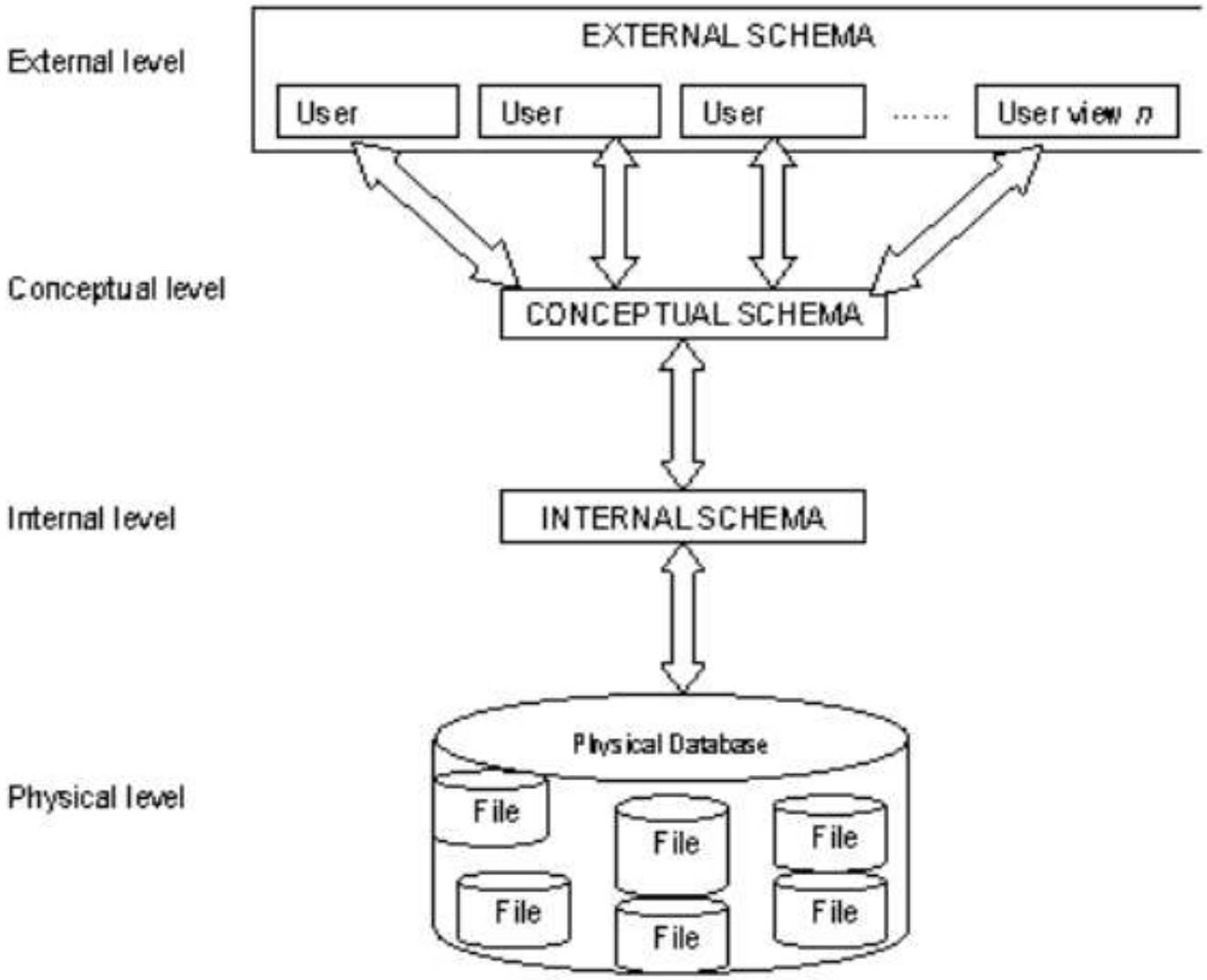
- The 3-Tier architecture contains another layer between the client and server. In this architecture, client can't directly communicate with the server.
- The application on the client-end interacts with an application server which further communicates with the database system.
- End user has no idea about the existence of the database beyond the application server. The database also has no idea about any other user beyond the application.
- The 3-Tier architecture is used in case of large web application.
- Advantages:
  - By separating the application into three distinct layers (presentation, application, and data), it allows for independent scaling, enhanced security protocols, and easier updates and modifycations
- Limitations:
  - increased complexity, potential performance bottlenecks due to increased network traffic,
  - higher development and maintenance costs, and reduced agility compared to simpler architectures


Database
Application Server
Server
Application Client
User
Client

Relational Database Management System (RDBMS)

240110102

Three Level ANSI-SPARC Database Architecture



Internal level or Storage level

- It is the physical representation of the database on the computer
- This level describes how the data is stored in the database
- It covers the data structures and file organizations used to store data on storage devices.
- It interfaces with the operating system access methods to place the data on the storage devices, build the indexes, retrieve the data and so on.
- Defined by the DBA
- The internal level is concerned with such things as:
  - Storage space allocation for data and indexes;
  - Record descriptions for storage (with stored sizes for data items)
  - Data compression and data encryption techniques

Conceptual Level or Logical level

- This level describes what data is stored in the database and the relationships among the data.
- This middle level in the three-tier architecture is the conceptual level which contains the logical structure of the entire database as seen by the DBA.
- It is a complete view of the data requirements of the organization that is independent of any storage considerations.
- The conceptual level represents:
  - All entities, their attributes, and their relationships;
  - Constraint on the data
  - Semantic information about the data
  - Checks to retain data consistency and integrity
  - Security information
- For instance, the description of an entity should contain only data types of attributes (for example, integer, real, character) and their length (such as the maximum number of digits or characters), but not any storage considerations, such as the number of bytes occupied

External Level or View level

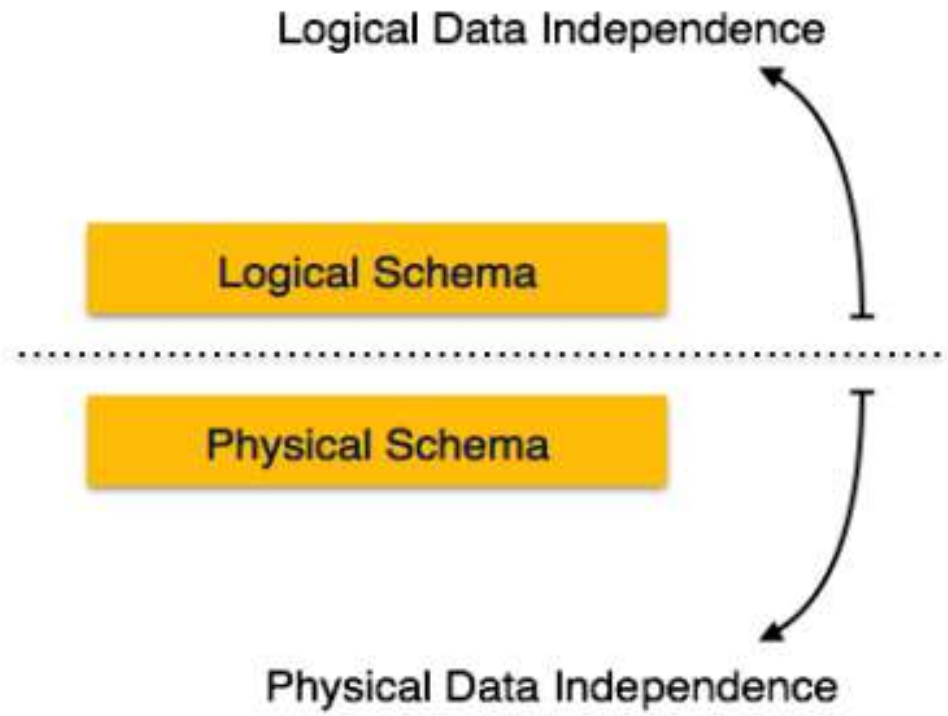
- It is the users' view of the database. This level describes that part of the database that is relevant to each user.
- External level is the one which is closest to the end users. .
- This level deals with the way in which individual users view data. Individual users are given different views according to the user's requirement.
- A view involves only those portions of a database which are of concern to a user. So, the same database can have different views for different users
- Example:
  - one user may view dates in the form (day, month, year),
  - while another may view dates as (year, month, day).

Advantages of Using Three-Tier Architecture

- It makes the logical separation between business layer and presentation layer and database layer.
- Migration to new graphical environments is faster.
- As each tier is independent it is possible to enable parallel development of each tier by using different sets of developers.
- Easy to maintain and understand large project and complex project.
- Since application layer is between the database layer and presentation layer so the database layer will be more secured and client will not have direct access to the database.
- Posted data from presentation layer can be verified or validated at application layer before updating it to the database.
- Database Security can be provided at application layer.
- Application layer or middle layer or business layer can be a protection shield to the database.
- New rules or new validation rules can be defined any time and changes made to middle layer will not affect presentation layer.
- We can hide unnecessary methods from business layer in the presentation layer.
- Easy to apply object oriented concept
- Easy to update data provider queries.

Data Independence

- Denotes the property that higher levels of abstraction and are not influenced by changes in the lower levels.
- The ability to change the schema at one level of a database system without affecting a schema definition in the next higher level is called data independence.
- There are three levels of abstractions so there are two types of data independence.
- There are two kinds of data independence:
  - Logical data independence
  - Physical data independence



Logical data independence

- Capacity to change the conceptual schema without having to change external schemas or application programs
- We may change the conceptual schema to expand the database (by adding a record type or data item), or to reduce the database (by removing a record type or data item) – here external schemas that refer only to the remaining data should not be affected
- Changes to constraints can be applied also to the conceptual schema without affecting the external schemas or application programs

Physical Data Independence

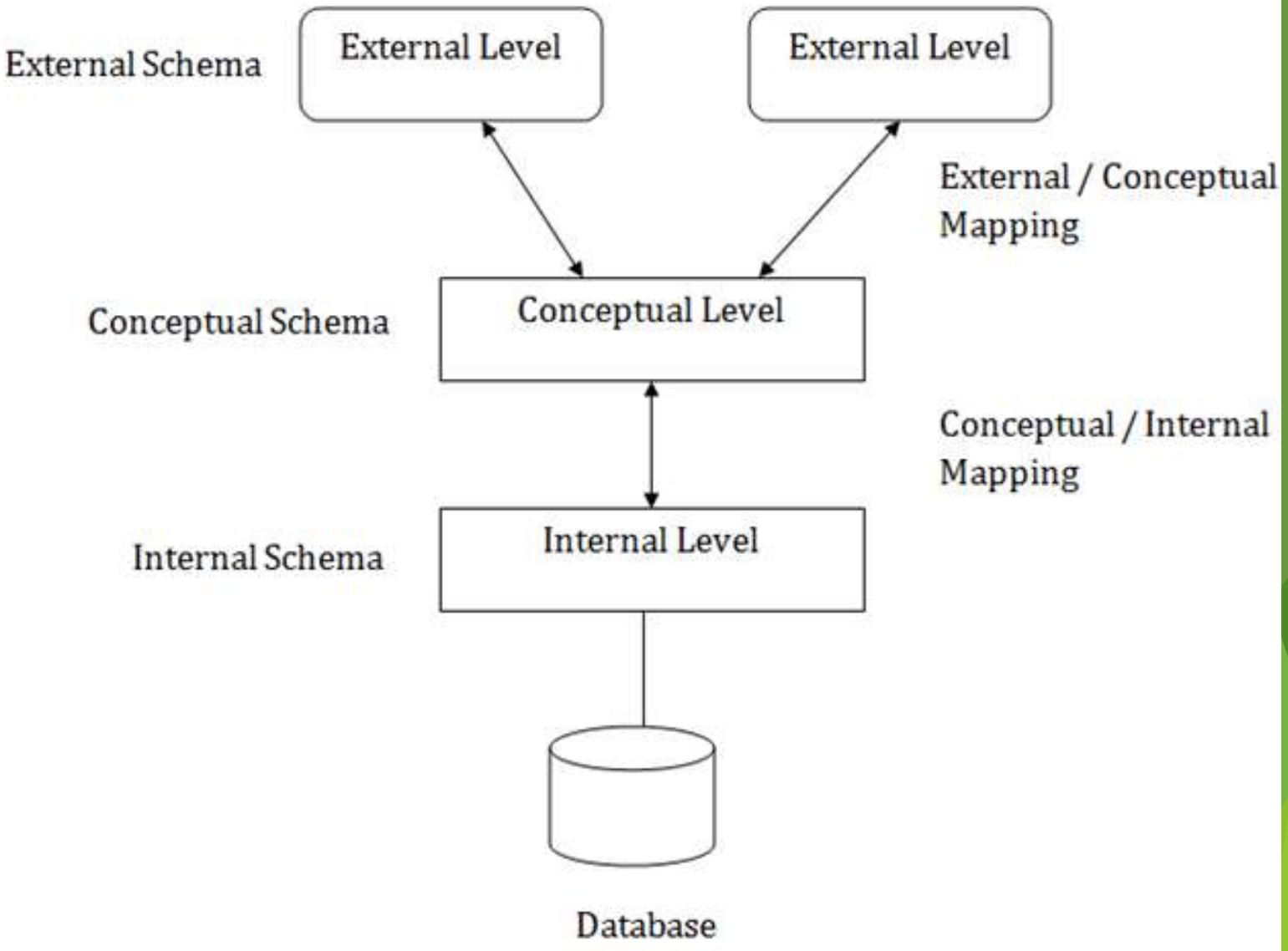
- Physical data independence is the capacity to change the internal schema without having to change the conceptual (or external) schemas
- For example, by creating additional access structures—to improve the performance of retrieval or update. If the data remains same in the database as before, we should not have to change the conceptual schema.

Data Abstraction

- The system hides certain details of how the data are stored and maintained is called data abstraction
- Since database system users are not computer trained, developers hide the complexity from users through 3 levels of abstraction, to simplify user's interaction with the system.

1. Physical level of data abstraction: The lowest level of abstraction which describes *how data are actually stored*.
2. Logical level of data abstraction: This level hides *what data are actually stored* in the database and what relationship exists among them.
3. View Level of data abstraction: View provides security mechanism to *prevent user from accessing certain parts of database*.

Mapping


External Schema
External Level
External Level
External / Conceptual Mapping
Conceptual Schema
Conceptual Level
Conceptual / Internal Mapping
Internal Schema
Internal Level
Database

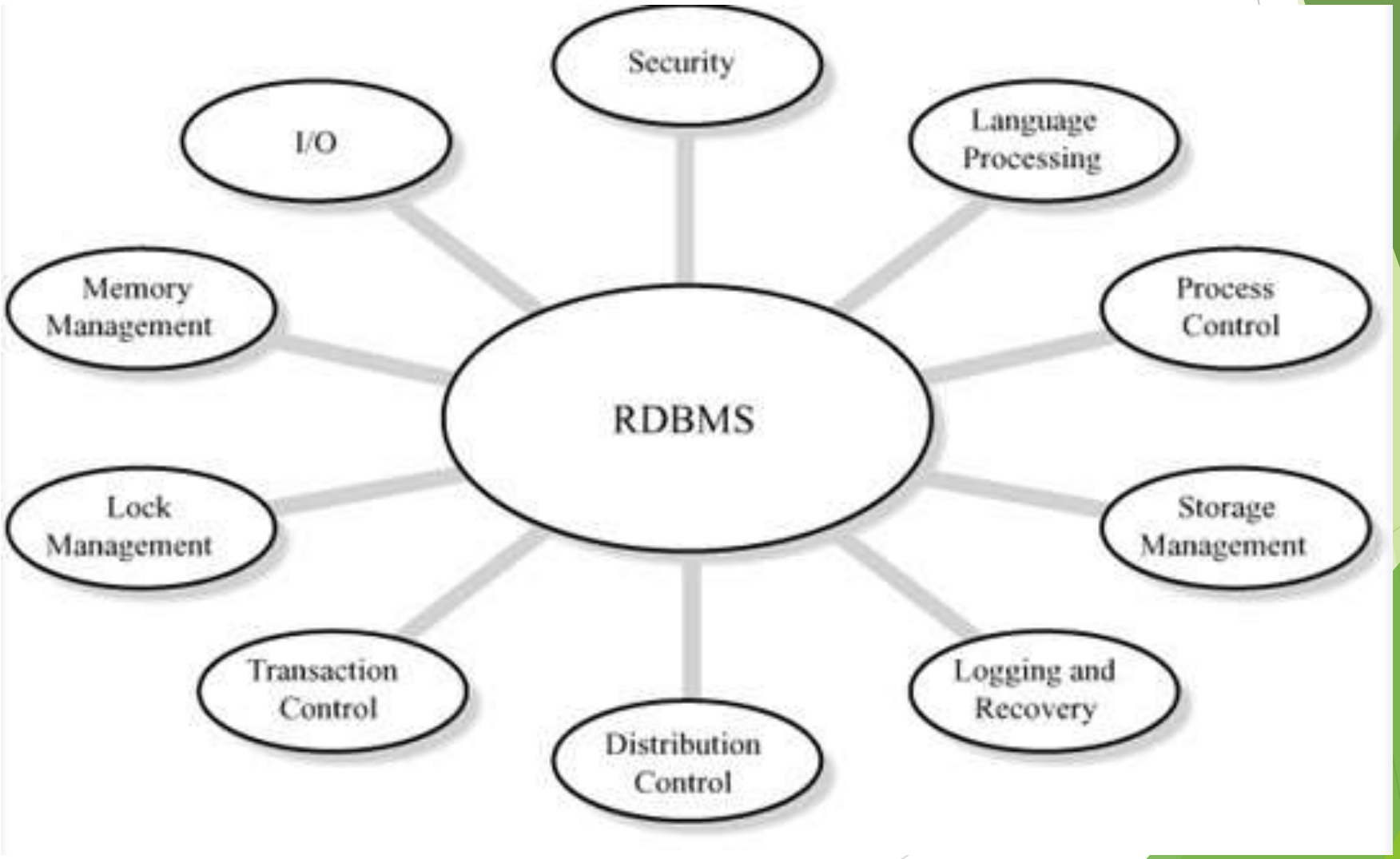
Relational Database Management System (RDBMS)

240110102

Relational Vs. Non Relational DBMS

Relational	Non-Relational
Data stored in structured format	No fix structure
Store and retrieval of data is easy	Not easy to manipulate data
Can use SQL	No fix language, it depends on the data storage format
Data stored in form of rows and column (relation)	Data in form of document , column-oriented, key-value store and graph
Any application can use the database	These types are used for a single application.
Can define key for unique identification of record, integrity constraints can be impose	There are no tables, rows, primary keys or foreign keys
Most popular relational databases have been Microsoft SQL Server, Oracle Database, MySQL and IBM DB2. Several free versions of these RDBMS platforms have gained popularity over the years, such as SQL Server Express, PostgreSQL, SQLite, MySQL and MariaDB.	Popular NoSQL databases are MongoDB, Apache Cassandra, Redis, Couchbase and Apache HBase

Benefits of using RDBMS



Benefits of using RDBMS

- Possible to design complex data storage and retrieval systems with ease (and without programming).
- Support for ACID transactions
  - Atomic
  - Consistent
  - Independent
  - Durable

- Easy for backup and recovery.
- Easy accessibility and query based protocols ensure good interactivity.
- Security and privacy ensured by maintenance of a key (Primary / Foreign key)
- Supporting multiple format file structures with multi level indexing.

Types of Databases

- Text/ file databases
  - A large portion of the information available worldwide in electronic form is actually in text form (other popular forms are structured and multimedia information).
- Relational database
  - Microsoft SQL Server, Oracle Database, MySQL and IBM DB2.
- NoSQL database
  - many different types of NoSQL databases, with different specifications: Key value, graph etc…
  - MongoDB is a most popular NoSQL Database (document-oriented database) that stores JSON-like documents in dynamic schemas
  - Cassandra DB is also highly scalable. Facebook® created Cassandra DB
  - The Firebase Realtime Database is a cloud-hosted NoSQL database
- Cloud database
  - 5 Best Cloud Databases to Use in 2021. Amazon DynamoDB, Google BigQuery, Azure SQL Server, Azure Cosmos DB, Amazon Redshift
- Object-oriented database
  - An object-oriented database is based on object-oriented programming, so data and all of its attributes, are tied together as an object. Object-oriented databases are managed by object-oriented database management systems (OODBMS). These databases work well with object-oriented programming languages, such as C++ and Java
  - Examples: Wakanda, ObjectStore
- Hierarchical database
  - Hierarchical databases use a parent-child model to store data. If you were to draw a picture of a hierarchical database, it would look like a family tree, with one object on top branching down to multiple objects beneath it.
  - Examples: IBM Information Management System (IMS), Windows Registry

Components of DBMS

1. Database Engine	The core service for accessing and processing data. It handles storage, query processing, and transaction management.
2. Data Definition Language (DDL) Compiler	Interprets and processes DDL statements like CREATE, ALTER, DROP to define schema and structure of the database.
3. Data Manipulation Language (DML) Compiler	Translates DML statements (INSERT, UPDATE, DELETE, SELECT) into low-level instructions the DBMS engine can execute.
4. Query Processor	Analyzes, parses, and optimizes SQL queries before execution. Converts high-level queries into efficient execution plans.
5. Transaction Manager	Ensures ACID properties (Atomicity, Consistency, Isolation, Durability) during transactions. Manages concurrency and crash recovery.
6. Storage Manager	Manages storage space on disk and memory. It handles data files, indexes, and data buffering for optimized access.
7. Buffer Manager	Controls data transfer between disk and main memory (RAM). Keeps frequently used data in memory for faster access.
8. Authorization and Integrity Manager	Enforces user access controls and data validation rules. Ensures only authorized users can access or modify the data.
9. Catalog/Metadata Manager	Maintains metadata — data about database structure like tables, columns, data types, indexes, constraints, etc.
10. User Interface or Query Interface	Allows users to interact with the DBMS. Interfaces may include command-line (e.g., SQL prompt) or graphical tools (e.g., MySQL Workbench).

DBMS Languages

- In a **Database Management System (DBMS)**, special languages are used to perform tasks such as defining the structure of data, manipulating data, and controlling user access. These are called **DBMS languages** and can be categorized into:
  - DDL – Data Definition Language
  - DML – Data Manipulation Language
  - DCL – Data Control Language

DDL – Data Definition Language

- DDL is used to define the structure of database objects such as tables, schemas, indexes, and views.
- DDL statements do not manipulate data.
- They only define or change the structure of database objects.
- Common DDL Commands:
  - Create
  - Alter
  - Drop
  - Truncate

DML – Data Manipulation Language

- DML is used to manipulate and manage data in the database.
- These commands allow inserting, updating, deleting, and retrieving data.
- Common DML Commands:
  - Select
  - Insert
  - Update
  - Delete

DCL – Data Control Language

- DCL is used to control access to data and database objects. It manages user privileges and permissions.
- Common DCL Commands:
  - Grant
  - Revoke

Traditional RDBMS

- Based on **Relational Model** (tables, rows, columns).
- Use **Structured Query Language (SQL)**.
- Great for **structured data**
- Strong **ACID** (**A**tomicity, **C**onsistency, **I**solation, and **D**urability) compliance.
- Examples: Oracle, MySQL, SQL Server, PostgreSQL

NoSQL

- **NoSQL** = “Not only SQL” – a broad class of **non-relational** databases designed for:
- High **scalability**
- Flexible **schema-less** structure
- **Unstructured/semi-structured data**

🔍 **Key Characteristics:**

- Horizontal scalability (easier distributed architecture)
- Schema flexibility (can change structure anytime)
- High performance for specific workloads
- For better performance and availability use **BASE:** Basically Available, Soft state, Eventually consistent (NOT ACID)

Types of NoSQL Databases:

Type	Description	Example
Document-based	Store data in JSON-like documents	MongoDB, CouchDB
Key-Value Stores	Simple key-value pairs	Redis, DynamoDB
Column-based	Store data in columns for fast retrieval	Cassandra, HBase
Graph-based	Store data as nodes and relationships	Neo4j, ArangoDB

When is NoSQL Used?

- Real-time analytics
- IoT applications
- Social media platforms
- Content management systems
- Applications with massive amounts of unstructured data

NewSQL

- NewSQL refers to modern relational databases
- Use SQL as query language
- Provide ACID compliance
- Offer scalability like NoSQL
- NewSQL = Best of both worlds: SQL + Scalability

Why NewSQL?

- To address scalability limitations of traditional RDBMS without giving up SQL features or transaction integrity.
- Designed for cloud-native applications.

Key Features of NewSQL:

- **Distributed architecture** for high availability
- Full **SQL support** (joins, indexes, ACID)
- **In-memory capabilities** for speed
- **Auto-sharding** (sharding refers to the practice of horizontally partitioning a large database into smaller, more manageable pieces called shards, which are distributed across multiple servers) and **replication** (process of creating and maintaining multiple copies of data across different nodes or servers)

Popular NewSQL Databases

Database	Features
Google Spanner	Global distribution, strong consistency
CockroachDB	Fault-tolerant, scalable SQL
VoltDB	In-memory processing for speed
NuoDB	Cloud-native, elastic scaling
TiDB	MySQL compatible, supports hybrid workloads

Comparison: RDBMS vs NoSQL vs NewSQL

Feature	RDBMS	NoSQL	NewSQL
Data Model	Relational	Non-relational	Relational
Query Language	SQL	Varies	SQL
Scalability	Vertical	Horizontal	Horizontal
ACID	Yes	No (BASE)	Yes
Schema	Fixed	Flexible	Flexible
Best Use	Structured data	Big data, IoT, social apps	Cloud apps needing SQL + scale

NoSQL = flexible, fast, scalable → good for modern apps with unstructured data.
NewSQL = power of SQL with scalability → best for enterprises needing reliability + scale.